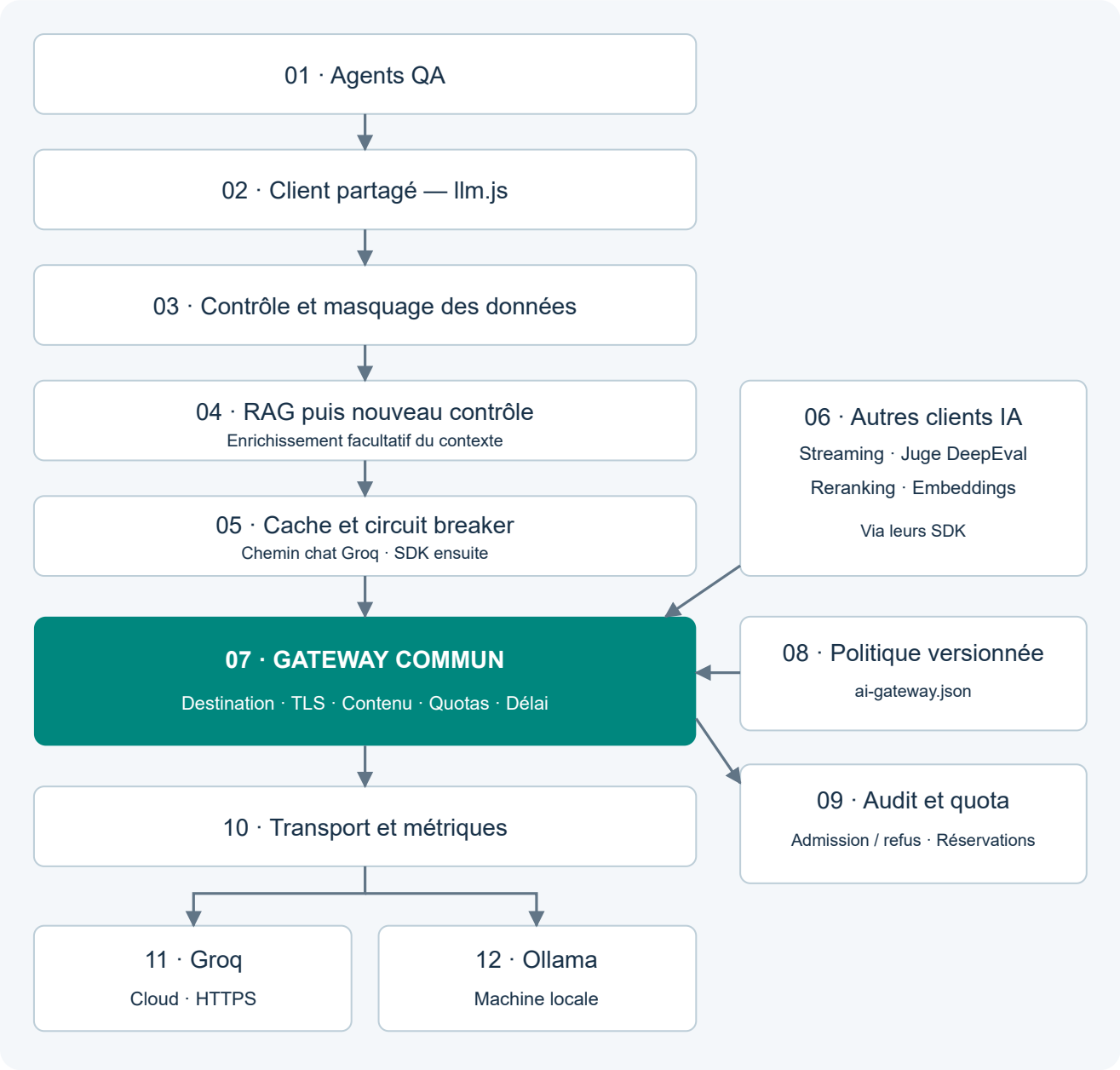
Gateway de sécurité des agents IA

Contrôler les données, les destinations et les budgets avant chaque appel aux fournisseurs IA.

Présentation client · Version 1.0 · 1 octobre 2026



Principe de sécurité : un refus bloque la requête avant l'envoi. Il ne déclenche pas de secours vers un autre fournisseur. Le schéma représente le flux aller ; une réponse en cache évite le réseau.

Du besoin QA au contexte contrôlé

01 · Agents QA

Ils exécutent les tâches du framework : analyse des échecs, audit qualité, génération de code, préparation de recommandations et reporting. Chaque agent sélectionne les preuves utiles à sa mission.

Responsabilité : limiter les données sélectionnées et interpréter la réponse selon les règles métier. Le gateway ne remplace pas cette validation.

`scripts/agents/*-agent.js`

02 · Client LLM partagé

Interface commune qui prépare les appels, choisit Groq ou Ollama et expose les modes texte, JSON, outils et streaming. Sur le chat, il déclenche les contrôles avant le RAG et avant le cache.

Mode local : AI_LOCAL_ONLY=true sélectionne Ollama pour le chat ; le gateway refuse les clients cloud.

`scripts/agents/llm.js`

03 · Contrôle et masquage des données

Les secrets connus et les formats de clés reconnus provoquent un refus. Les emails, identifiants explicites, cookies et autorisations reconnus sont masqués. Les URL HTTP(S) contenues dans les données sont anonymisées, sans chemin, paramètres ni fragment.

Couverture : objets imbriqués et arguments d'outils JSON. Le traitement ne modifie pas l'URL réseau du fournisseur.

`shared/ai-gateway.js · shared/payload-privacy.js`

04 · Enrichissement RAG

Le RAG recherche des éléments de connaissance pertinents et les ajoute au contexte de la demande. Cet enrichissement est facultatif. Le contexte obtenu passe à nouveau par le contrôle avant d'être utilisé.

Point d'attention : les documents retrouvés restent des données non fiables ; leur présence ne vaut pas autorisation de suivre leurs instructions.

`shared/hybrid-rag.js`

05 · Cache et circuit breaker

Sur le chemin chat Groq, le cache réutilise une réponse existante et évite un appel réseau. Le circuit breaker suspend temporairement les appels après des échecs répétés. Les réponses Groq sont contrôlées avant stockage et les réponses du cache sont contrôlées à la lecture.

Secours : une panne ordinaire peut conduire à Ollama ; un refus de sécurité ne le permet pas. Le streaming ne suit pas ce chemin de cache.

`shared/circuit-breaker.js · llm.js`

Un contrôle partagé avant le réseau

06 · Autres clients IA

Le streaming diffuse progressivement les réponses. Le juge DeepEval évalue les réponses IA. Le reranking réordonne les documents retrouvés. Les embeddings transforment le texte en vecteurs pour la recherche. Leurs transports passent aussi par le gateway.

07 · Gateway commun

Point de contrôle avant le transport : il vérifie le fournisseur, l'origine, la route et le modèle autorisés ; refuse les redirections et TLS désactivé ; inspecte et nettoie le contenu ; contrôle taille, plafond de sortie, quota et délai.

Un refus porte le code **AI_GATEWAY_BLOCKED**. Un audit indisponible ou un quota invalide bloque également l'appel.

`shared/ai-gateway.js`

08 · Politique versionnée

Fichier central définissant les fournisseurs, destinations, modèles et limites autorisés. Il permet de rendre les règles explicites et de les examiner lors des revues de changement.

`config/ai-gateway.json`

09 · Audit et réservations de quota

L'audit conserve l'admission ou le refus, le motif et les métadonnées de la demande, sans prompts ni réponses. Un registre partagé entre processus réserve le budget de chaque tentative, y compris les reprises du SDK.

Interprétation : ALLOW signifie « admis avant le réseau », pas « appel fournisseur réussi ».

`logs/gateway/audit.jsonl · logs/gateway/quota.json`

10 · Transport et métriques

Le transport exécute uniquement les requêtes admises. La couche de mesure relève l'usage et le résultat HTTP sans conserver le contenu des prompts ou des réponses dans ces métriques.

`shared/task-metrics.js · meteredFetch`

11 · Groq

Fournisseur cloud accessible par HTTPS. Il reçoit le contenu autorisé et nettoyé. Les modèles sont limités par la politique.

12 · Ollama

Exécute les modèles sur la machine locale. Les adresses de boucle locale sont autorisées ; HTTP est permis uniquement pour ce service local.

EXPLOITATION · GARANTIES ET LIMITES

Ce que protège cette architecture

Contrôle	Politique actuelle
Taille du corps de requête	200 000 octets maximum
Génération	4 096 tokens maximum ; une complétion par appel
Cadence	60 requêtes par minute, partagées dans le workspace
Délai de transport	120 secondes par requête ; annulation prise en compte
Budget par tâche	1 000 000 d'unités réservées sur 24 h : octets d'entrée + plafond de sortie. Ce n'est ni un comptage de tokens ni un plafond financier.
Mode strictement local	AI_LOCAL_ONLY=true : appels cloud refusés par le gateway

Exemple de parcours

Un agent analyse un échec contenant une adresse email et une URL interne. Ces éléments sont masqués. Le contexte enrichi est contrôlé à nouveau. Si la destination et le budget sont autorisés, le transport envoie la requête nettoyée. Si une clé privée ou un secret connu est détecté, l'appel est bloqué.

> **Deux niveaux distincts :** le filtrage protège les données incluses dans le message ; la liste des destinations autorisées contrôle à quel service le message peut être envoyé.

Limites à présenter au client

- Protection applicative des clients IA raccordés : ce n'est pas un pare-feu, un proxy réseau ni une frontière contre du code local hostile.
- Le masquage ne reconnaît pas toutes les données personnelles, secrets encodés ou informations confidentielles. La sélection des sources autorisées reste nécessaire.
- Le gateway ne garantit ni l'absence d'injection de prompt ni la justesse du verdict IA. Les validations métier et humaines restent distinctes.
- Les journaux, mémoires et caches historiques ne sont pas nettoyés rétroactivement. L'anonymisation peut nécessiter une revue locale des preuves citées.

Éléments de vérification

La dernière validation locale rapportée comprend 37 tests distincts réussis, avec des transports simulés et sans appel réel à Groq. Elle couvre notamment les refus de destination, TLS, secrets, quotas, streaming et masquage. Les contrôles TypeScript et Cucumber dry-run ont également réussi. Ces vérifications ne constituent pas un audit de sécurité indépendant.

Références internes : docs/ai-gateway.md ; config/ai-gateway.json ; scripts/agents/llm.js ; scripts/agents/shared/ai-gateway.js ; payload-privacy.js. Document descriptif de la configuration examinée, sans données client ni secrets.